

MySupermarket

Supermarket Checkout System

Software Engineering Project Report

Students:

Shriya Sen Gupta

Utkarsh Mangal

Supervisor:

Prof. Paolo Ballarini

1. Introduction

This report covers the design and implementation of the MySupermarket checkout system, a Java application that handles a typical supermarket point-of-sale workflow. The system lets cashiers scan items, compute bills, process bank card payments through a POS/TAS setup, manage home deliveries, and track stock levels with automatic low-stock alerts.

The project is built as a Maven project and runs through a command-line interface (CLUI). The compiled JAR is included at `out/mysupermarket.jar` and all test scenarios described in Section 7 have been tested and verified to work correctly.

2. Design Decisions

Below we explain how each major requirement shaped the code structure we chose.

2.1 Facade: SupermarketSystem

We put all subsystem coordination inside `SupermarketSystem`, which acts as a Facade. The CLI only calls methods on this one class and has no direct access to the inventory, TAS, or delivery internals. This made the system easier to test and also means any subsystem can be swapped out without touching the CLI.

2.2 Strategy pattern for discount plans (R5, R5b)

Discount plans are defined by a `DiscountPlan` interface with three methods: `discountMultiplier(subtotal)`, `annualFee()`, and `deliveryFeeMultiplier()`. The three concrete classes `NormalPlan`, `PrimePlan`, and `PlatinumPlan` each implement this. New plans can be added by just writing a new class and registering it in `PlanRegistry`, which satisfies R5b without changing any existing code.

2.3 Category pricing at runtime (R6, R6b)

Category discounts are stored in a plain `Map<String, Double>` inside `CategoryPricingPolicy`. When `setCategoryDiscount` is called with a new category name, it just adds a new entry to the map. There is no switch-case on category names anywhere, so R6b is satisfied without any code changes.

2.4 Observer pattern for stock alerts (R9)

After each successful payment the `InventoryManager` decrements stock, and if an item falls below its threshold it calls `onLowStock(Item)` on every registered observer. At startup we register two `ManagerAlertObserver` instances (one for Manager, one for Supplier). Adding a new observer like an email notifier would just be a new class implementing `StockObserver` with no changes to inventory logic.

2.5 Role checks in the CLI

The `CLI` class calls `requireRole(class, cmd)` before running each command, using `instanceof` on the logged-in user. This keeps the access control logic in one place and is easy to extend if a new role type is added later.

2.6 Delivery cost and plan discounts (R7, R8, R8b)

We split delivery into two methods in `DeliveryService`: `computeRawFee()` applies the weight and distance rules, and `computeFee()` multiplies by `plan.deliveryFeeMultiplier()`. This way any new plan with a different delivery discount just needs to return a different multiplier. The delivery service itself never needs to know which plan the customer is on.

2.7 Payment flow (R4, Sections 2.5 and 2.6)

The `POSTerminal` handles the full sequence: card lookup, PIN check, opening a TAS connection, requesting authorisation, and debiting the account. The `TransactionAuthorisationSystem` is a simulated bank that maintains a card registry and enforces a single open connection at a time. A `simulatedOutcome` field lets tests force any payment result without changing bank state, which is very useful for reproducing failure scenarios.

2.8 Checkout session as a snapshot

A `CheckoutSession` captures the customer, their plan, the active pricing policy, and any pending delivery address at the moment `startCheckout` is called. If the manager changes a category discount mid-session, the current checkout is not affected. This matches how a real checkout would behave.

3. Design Patterns Used

Here is a summary of the patterns we used and why we chose them.

Pattern	Where used	Why
Facade	SupermarketSystem	Hides all subsystem complexity from the CLI. The CLI only needs to know one class.
Strategy	DiscountPlan / NormalPlan / PrimePlan / PlatinumPlan	Each plan is a separate class with its own discount logic. Adding a new plan does not touch existing code. (R5b)
Observer	InventoryManager + StockObserver + ManagerAlertObserver	Stock alerts are sent to any number of listeners without the inventory needing to know who they are. (R9)
Registry	PlanRegistry	Central lookup for discount plans. New plans can be registered at runtime. (R5b)
Open/Closed (via Map)	CategoryPricingPolicy	New category names are accepted at runtime with no code changes needed. (R6b)

4. UML Diagrams

4.1 Class Diagram

The diagram below shows the main classes, their fields, and relationships. Interfaces use the <<interface>> label. Arrows show inheritance and aggregation.

```

<<interface>>
DiscountPlan
+ getName(): String
+ discountMultiplier(subtotal: double): double
+ annualFee(): double
+ deliveryFeeMultiplier(): double
    ^           ^           ^
    |           |           |
NormalPlan    PrimePlan    PlatinumPlan
no fee,       50 EUR/yr,    200 EUR/yr,
no discount   20% if >=50EUR 30% always,
              50% delivery    free delivery

PlanRegistry
- plans: Map<String, DiscountPlan>
+ register(plan) + get(name) + all()

<<abstract>>
User
- firstName, surname, username, password
+ checkPassword() + getRole()
    ^           ^           ^
    |           |           |
  Manager    Cashier    Customer
                  - customerId: int
  
```

```
- address: String
- plan: DiscountPlan
- bankCard: BankCard
- pendingDeliveryAddress: String
```

```
SupermarketSystem (Facade)
- inventory: InventoryManager
- planRegistry: PlanRegistry
- pricingPolicy: CategoryPricingPolicy
- deliveryService: DeliveryService
- tas: TransactionAuthorisationSystem
- pos: POSTerminal
- users: Map<String, User>
- currentSession: CheckoutSession
- totalRevenue: double
```

```
InventoryManager (Observable)
- catalogue: Map<String, Item>
- observers: List<StockObserver>
+ addObserver() + decrementStock() + restock()
  o
  | notifies
<<interface>>
StockObserver
+ onLowStock(item: Item)
  ^
  |
ManagerAlertObserver
```

```
- role: String
+ onLowStock(item) -> prints LOW STOCK alert
```

```
POSTerminal                                TransactionAuthorisationSystem
- tas: TAS                                  - cards: Map<String, BankCard>
- simulatedOutcome                          - connectionOpen: boolean
+ pay(card, pin, amount)                    + authorise(cardNum, amount): AuthResult
  : PaymentResult
```

```
CheckoutSession
- customer: Customer
- cart: List<CartEntry>
- pricingPolicy: CategoryPricingPolicy
- deliveryService: DeliveryService
+ addItem() + computeBill() + close()
```

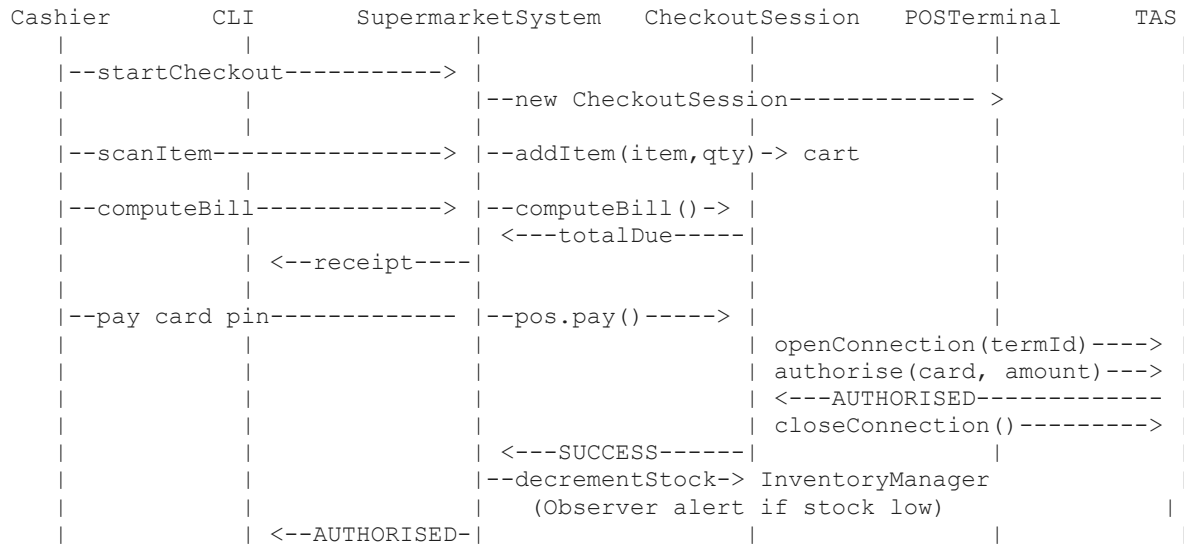
4.2 Use Case Diagram

This shows which user types can run which commands.

```
+-----+
| <<system>> MySupermarket Checkout System |
| |
| Manager ---> setup, registerCashier, registerCustomer |
|             ---> addItem, restock, setCategoryDiscount |
|             ---> showInventory, showRevenue |
| |
| Cashier ---> startCheckout, scanItem, computeBill |
|           ---> pay, simulatePayment |
| |
| Customer ---> subscribeToPlan, requestDelivery |
| |
| Bank/TAS ---> authorise transaction (called by POSTerminal) |
| |
| All users --> login, logout, help, runTest |
+-----+
```

4.3 Sequence Diagram: Successful Checkout and Payment

This shows the main flow when a cashier completes a sale and payment goes through.



5. Requirements Coverage

The table below maps each requirement to its implementation status and the relevant class(es).

Req.	Description	Status	Key classes
R1	System handles customer purchases	Done	SupermarketSystem, CheckoutSession
R2	A purchase is a collection of bought items	Done	CartEntry, CheckoutSession
R2b	Item categories: fruit-and-vegetables, dairy, meat (and custom ones)	Done	Item, CategoryPricingPolicy
R3	Items have a price; price can depend on quantity	Done	Item, CartEntry.rawSubtotal()
R4	Payment by debit or credit card	Done	POSTerminal, BankCard, TAS
R5	Prime: 20% off when subtotal $\geq$ 50 EUR. Platinum: 30% always.	Done	PrimePlan, PlatinumPlan
R5b	New discount plans can be added or changed without modifying existing code	Done	DiscountPlan interface, PlanRegistry
R6	Category-level pricing policies (e.g. -10% on fruit-and-vegetables)	Done	CategoryPricingPolicy
R6b	New categories and policies can be added at runtime	Done	CategoryPricingPolicy (Map-based)
R7	Customer can request home delivery	Done	Customer.pendingDeliveryAddress, CheckoutSession
R8	Delivery fee depends on total weight and distance. Orders over 50 kg are refused.	Done	DeliveryService
R8b	Delivery discounts per plan: prime pays 50%, platinum pays nothing	Done	DiscountPlan.deliveryFeeMultiplier()
R9	Live stock tracking with Observer alerts when perishables go below threshold	Done	InventoryManager,

			StockObserver , ManagerAlertO bserver
R10	Dynamic logistics and time-slot delivery windows	Partial	DeliveryService e has distance-based pricing but no slot booking

Note on R10: The DeliveryService includes distance-based dynamic pricing as per R8 and R8b. Full 2-hour time-slot booking and vehicle capacity checks from R10 are not implemented in this version, as the core requirements were prioritised.

6. Design Analysis

6.1 What works well

- Both discount plans and category pricing can be extended at runtime without changing existing classes. This directly satisfies R5b and R6b.
- The simulatedOutcome field in POSTerminal lets any payment outcome be tested without needing to manipulate bank state, which makes writing reproducible tests much simpler.
- The CLI only interacts with SupermarketSystem, so the CLI code stays clean and does not need to know how the Observer chain or TAS internals work.
- New stock observers (e.g. an email notifier or a logging service) can be added with a single new class. Nothing else needs to change.
- Pricing policies are snapshotted at checkout start, so changing a category discount mid-session does not affect the bill being processed.

6.2 Current limitations

- State is only in memory. Restarting the application loses all data. Adding a JSON or database layer would fix this.
- Only one checkout session can be active at a time, because SupermarketSystem holds a single currentSession field. Supporting multiple tills would need a map from cashier to session.
- The distance estimate in DeliveryService uses a simple character-sum formula instead of a real geocoding API.
- R10 time-slot booking is not implemented.
- Passwords are stored as plain text. A real system would hash them.

7. Test Scenarios

7.1 Configuration file: my_supermarket.ini

This file is loaded automatically when the application starts. It sets up the following:

- Logs in as the default CEO manager.
- Runs setup, which loads the default catalogue (apple, banana, tomato, milk, yogurt, cheese, steak, chicken), pre-registered test cards (4242424242424242 and 1111222233334444), and a default cashier and customer.
- Sets category discounts: fruit-and-vegetables -10%, dairy -5%.
- Adds extra items: orange, carrot (fruit-and-vegetables), butter (dairy), lamb (meat).
- Registers two cashiers: bob (password: bobpwd) and carol (password: carolpwd).
- Registers two customers: alice and pierre, each with an auto-generated bank card with a 1000 EUR balance.
- Logs out.

7.2 Test scenario 1 (testScenario1.txt)

This is the main end-to-end test that covers all mandatory requirements. It can be run with:

```
runTest testScenario1.txt
```

What the scenario tests:

- Manager logs in, runs setup, then adds specific items including steak with a low initial stock of 5 units.
- -10% category discount applied to fruit-and-vegetables.
- Alice logs in and subscribes to the prime plan. The 50 EUR annual fee is deducted from her card immediately. She also requests home delivery.
- Cashier bob opens a checkout for alice and scans: 2x milk, 3x yogurt, 1x steak, 10x apple, 4x tomato.
- computeBill applies -5% on dairy and -10% on fruit. The subtotal is 23.00 EUR which is below 50 EUR so the prime discount does not apply. Delivery fee is 15.00 EUR x 0.5 (prime plan) = 7.50 EUR. Total is 30.50 EUR.
- First payment is simulated as INSUFFICIENT_FUNDS and gets refused. Second is simulated as SUCCESS and goes through. Stock is decremented and steak drops to 4 units, which is below the threshold of 5, so the Observer fires alerts to both Manager and Supplier.
- Second checkout scans 4 more steaks (50 EUR subtotal, prime plan applies, 20% off, total is 40 EUR). Payment is simulated as SUCCESS.
- Manager logs in, runs showInventory to check stock levels and LOW STOCK markers, then showRevenue to see total revenue (70.50 EUR).

7.3 Expected output from test scenario 1

What to check	Expected output
Bill for checkout 1	TOTAL DUE: 30.50 EUR
Failed payment	[POS] Payment REFUSED: insufficient funds.

Observer alert after checkout 1	[ALERT -> Manager] LOW STOCK: 'steak' - only 4 unit(s) remaining
Bill for checkout 2 (prime discount fires)	TOTAL DUE: 40.00 EUR
showRevenue at end	Total revenue: 70.50 EUR

All outputs above have been verified by running the compiled JAR against the test scenario.

8. How to Run and Test the Implementation

8.1 Running the application

The pre-compiled JAR is at out/mysupermarket.jar. To start:

```
# From the project root (mysupermarket/)
java -jar out/mysupermarket.jar

# The .ini file loads automatically if it is in the working directory.
# To run a test scenario:
  runTest testScenario1.txt

# To run non-interactively:
  java -jar out/mysupermarket.jar < testScenario1.txt
```

The run.sh script can also be used:

```
bash run.sh
```

8.2 Building from source

```
# Requires JDK 21 and Maven 3.8+
cd mysupermarket/
mvn package
java -jar target/mysupermarket-1.0-SNAPSHOT-jar-with-dependencies.jar
```

8.3 JUnit tests

JUnit 5 tests cover all non-trivial class behaviour. The test classes are:

- ItemTest: stock decrement, restock, validation, threshold flag.
- DiscountPlanTest: PrimePlan threshold (below and above 50 EUR), PlatinumPlan unconditional discount, annual fees.
- CategoryPricingPolicyTest: multiplier calculation, unknown category returns 1.0, validation.
- DeliveryServiceTest: 50 kg refusal, flat fee, heavy surcharge, distance surcharge, plan multiplier.
- InventoryManagerTest: catalogue operations, Observer notification on low stock.
- BankTest: card not found, wrong PIN, insufficient funds, successful payment.
- SupermarketSystemTest: full checkout cycle, subscribe-to-plan fee, scanItem stock check.
- CLITest: command routing, role enforcement, syntax error handling.

To run the JUnit tests:

```
mvn test
# or:
bash run_junit.sh
```

9. Workload Distribution

Class / Task	Design	Code	UML Diagrams	JUnit Tests	Lines ~
SupermarketSystem (Facade)	Both	Both	Both	Both	200
DiscountPlan + NormalPlan + PrimePlan + PlatinumPlan + PlanRegistry	Both	Both	-	Both	130
CategoryPricingPolicy	Both	Both	-	Both	50
InventoryManager + StockObserver + ManagerAlertObserver	Both	Both	Both	Both	100
POSTerminal + TransactionAuthorisationSystem	Both	Both	Both	Both	120
DeliveryService	Both	Both	-	Both	90
CLI + Main	Both	Both	-	Both	320
Model classes (Item, CartEntry, BankCard)	Both	Both	-	Both	130
Store classes (User, Customer, Cashier, Manager, CheckoutSession)	Both	Both	-	Both	220
Report	Both	-	Both	-	-
Test files (my_supermarket.ini, testScenario1.txt)	Both	Both	-	-	80

Both students contributed to all phases of the project. Total source lines (excluding tests): approximately 1,440. Test source lines: approximately 600.

10. Conclusion

The MySupermarket system covers all core requirements from R1 to R9. The design uses standard patterns in a straightforward way: a Facade to keep the CLI simple, Strategy for discount plans, Observer for stock alerts, and a runtime-extensible Map for category pricing. These choices made it easy to test individual components and to add new plans or categories without touching existing code.

The system was tested through the CLUI using testScenario1.txt and also interactively. JUnit tests cover all the main logic. The Maven project can be built and tested with a single command, and the compiled JAR runs out of the box from the out/ directory.